Chapter - 1
Learning From Data

Aviral Janveja

1 Introduction

Let us understand machine learning, using an example from the finance domain. Suppose a customer applies for a loan. Now, the bank must decide whether granting the loan is a good idea or not, since their goal is to maximize profit while minimizing risk. Naturally, the bank has no magic formula to predict whether a customer will make timely repayments or not. However, it is indeed possible to look at similar customers from the past, and check how their repayment behavior turned out to be.

The **idea** is to try and discover the underlying pattern that connects customer information to their repayment behavior, using past data points. Hopefully, we can then use it to predict the creditworthiness of fresh applicants. Using this example, let us now outline the main elements of the machine learning process :

1.1 The Underlying Pattern

As discussed above, in machine learning, our goal is to uncover the underlying pattern from a given set of data points. In the loan example, this underlying pattern is the relation that connects customer information to their repayment behavior. Although this relation is unknown to us, we do see glimpses of it through the available data points.

So, how do we describe this unknown pattern mathematically? Is it a deterministic function that always produces the same output for a given input, or is it something inherently probabilistic? To answer this, let us consider a simple practical question :

> **Can two loan applicants with identical information exhibit different repayment behaviours?**

Well, it is entirely possible and in fact quite common. Two applicants may have identical ages, salaries and employment histories, yet one eventually repays the loan while the other defaults. This can happen because of factors that are not captured in our data, such as job loss, unexpected medical expenses, or simply the inherent uncertainty in human behaviour. This leads us to an important observation :

> **Most real world scenarios are practically probabilistic rather than deterministic in nature.**

Hence, we will build our framework for machine learning accordingly, to capture that uncertainty from the very beginning. Instead of assuming a single correct output for every input, we describe the underlying pattern in terms of likelihood of different possible outcomes, given an input. Formally, this is represented as a conditional probability distribution, known as the **target distribution** :

$$P(y \mid \mathbf{x})$$

It is basically a function which gives us the probability of every possible output, for a given input.

Target Distribution Vs Target Function

To better understand target distribution, let us first consider a deterministic scenario. Suppose the bank follows a strict policy :

> **Approve every applicant whose annual income exceeds Rs. 7 lakh, otherwise reject the application.**

There is no uncertainty in this rule. Every input produces exactly one correct output. Such a relation can be represented simply, using a regular function. Let us call it the **target function** :

$$f(\mathbf{x})$$

For example, considering annual income as the only input for simplicity, if x = Rs. 9 lakhs per annum, then :

$$f(9) = +1$$

Indicating that the loan is approved. However, real world decision making is rarely this simple. As discussed above, even if two applicants have the same information, their repayment behaviour may still differ because our data cannot capture every influencing factor. That is why, instead of assigning a single output, we assign probabilities to the possible outcomes. For example :

$$P(y = +1 \mid x = 9) = 0.80$$
$$P(y = -1 \mid x = 9) = 0.20$$

Indicating that an applicant with a salary of Rs. 9 lakhs per annum has an 80% chance of repaying the loan and a 20% chance of defaulting. Finally, it is important to **note** that the deterministic target function can be considered a **special case** of the more general target distribution. Where, all of the probability is concentrated on a single output, while every other output has zero probability.

1.2 The Data Set

Although the underlying pattern is unknown to us, we do have examples generated from it in the form of our data set. In the loan approval example from above, each data point corresponds to a past customer and collecting N such data points gives us the required data set :

$$\mathbf{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3)...(\mathbf{x}_N, y_N)\}$$

Where $\mathbf{x}_n$ is an array of numbers describing the n^{th} customer, through quantities such as age, salary, years in residence, current debt and so on. The corresponding output $y_n \in \{+1, -1\}$ indicates whether that customer turned out to be creditworthy or not in hindsight. Now, these data points do not reveal the target distribution completely, but they provide us with examples from which we can attempt to approximate it. In other words, the data set acts as our limited window into the underlying pattern.

2 Our First Learning Model : The Perceptron

Linear classifiers such as the perceptron do not model the entire underlying target distribution. Instead, they optimize a linear decision boundary as per the labels assigned to the past data-points. It can be thought of as the perceptron approximating the target distribution, by learning the most probable label for a given x.

For example, let us say the underlying probability distribution given a point x is as follows :

$$P(y = +1 \mid \mathbf{x}) = 0.8$$

and

$$P(y = -1 \mid \mathbf{x}) = 0.2$$

The perceptron doesn't care about modeling this underlying probability distribution. It just gets the final label $y = +1$ and that is what the perceptron will try to learn and optimize for. This is because perception is a simple linear model. Further there will be other advanced models that will try to capture the probabilistic aspect explicitly.

Let us begin by looking at our data set, which is available to us in the form of N input-output pairs :

$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3)...(\mathbf{x}_N, y_N)$$

As discussed above, $\mathbf{x}_n$ is the application information of the n^{th} customer, which is represented by a column matrix as shown below :

$$\mathbf{x}_n = \begin{bmatrix} x_1 \\ x_2 \\ . \\ . \\ x_d \end{bmatrix}$$

Here x_1, x_2 and so on upto x_d are the attributes of a customer like age, salary, years in residence, outstanding debt and so on. Whereas $y_n \in \{+1, -1\}$ is the binary value representing the creditworthiness of a past customer in hindsight.

2.1 Perceptron Hypothesis Function

Now, what a perceptron basically does, is to assign weights to each of the customer attributes. These weights are multiplied to their respective attributes and then summed-up as follows :

$$w_1x_1 + w_2x_2 + ... + w_dx_d$$

The above linear summation of weights and attributes can be called the credit score of an individual customer. The idea is pretty simple, If the credit score is greater than a certain threshold, then we approve the loan, else we deny it :

$$w_1x_1 + w_2x_2 + ... + w_dx_d > \text{threshold (Approve Loan)}$$
$$w_1x_1 + w_2x_2 + ... + w_dx_d < \text{threshold (Deny Loan)}$$

This is equivalent to :

$$w_1x_1 + w_2x_2 + ... + w_dx_d - \text{threshold} > 0 \text{ (Approve Loan)}$$
$$w_1x_1 + w_2x_2 + ... + w_dx_d - \text{threshold} < 0 \text{ (Deny Loan)}$$

We can simplify the above expression further, by taking the minus of threshold as the zero$^{\text{th}}$ weight ($w_0 = -\text{threshold}$) and introducing an artificial attribute $x_0 = 1$ to go along with it, hence giving us :

$$w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d > 0 \text{ (Approve Loan)}$$
$$w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d < 0 \text{ (Deny Loan)}$$

Next, we re-write the above expression in vector (column matrix) notation to make it shorter and cleaner :

$$w_0x_0 + w_1x_1 + .. + w_dx_d = \begin{bmatrix} w_0 & w_1 & . & . & w_d \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ . \\ . \\ x_d \end{bmatrix} = \mathbf{w^T x}$$

The above formula outputs a real value that could be greater than, equal to or less than zero. In order to output a binary value, similar to y, we wrap the above formula inside a **sign** function.

$$\text{sign}(\mathbf{w^T x})$$

The sign is just a simple function that takes in a real number value and outputs $+1$ if the value is greater than zero and -1 if it is less than or equal to zero, hence giving us the final form of our perceptron hypothesis function.

2.2 Perceptron Learning Algorithm

Examining the perceptron hypothesis function, that we have arrived at above :

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x})$$

We see that $\mathbf{x}$ representing the customer information is already a given, along with y, which is the correct loan decision in hindsight. So, the underlying pattern that we are trying to learn will be reflected in our choice of weights, as represented by the weight vector $\mathbf{w}$. Hence, our task now is to arrive at a set of weights, that are able to classify all given data-points correctly.

So, how do we compute the correct weights? First, we initialize the weights randomly, let us say initialize them all to zero. The perceptron learning algorithm then iterates through the dataset, looking for misclassified points $(\mathbf{x}_n, y_n)$, where :

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) \neq y_n$$

Whenever such a misclassified point is found, the weight vector is updated by applying the following **update rule** :

$$\mathbf{w}_{new} = \mathbf{w} + y_n\mathbf{x}_n$$

This process continues, until no misclassified points remain in the dataset. And that is it, we then have our required set of final weight values.

2.3 Update Rule Justification

Let us examine how the above update rule actually nudges the perceptron in the right direction at each misclassified point. If a point $(\mathbf{x}_n, y_n)$ is misclassified, it means :

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) \neq y_n$$

Accordingly, the perceptron updates the weight vector as follows :

$$\mathbf{w}_{new} = \mathbf{w} + y_n\mathbf{x}_n$$

Substituting the updated weight vector from above and ignoring the sign function for now, we get the following :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = (\mathbf{w} + y_n\mathbf{x}_n)^\mathbf{T}\mathbf{x}_n$$

Expanding the right-hand side, we get :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + y_n\mathbf{x}_n^\mathbf{T}\mathbf{x}_n$$

This is equivalent to :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + y_n||\mathbf{x}_n||^2$$

Therefore, for a misclassified point, where $y_n = +1$ and $\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) = -1$, it implies $\mathbf{w}^\mathbf{T}\mathbf{x}_n \leq 0$ and after the update we get :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + ||\mathbf{x}_n||^2$$

The update is adding some positive value to $\mathbf{w^T x}_n$ which might make $\mathbf{w^T x}_n > 0$ as required. Although, it does not guarantee an immediate sign correction, it clearly nudges the classifier in the right direction.

Similarly, when $y_n = -1$ and $\text{sign}(\mathbf{w^T x}_n) = +1$, implying $\mathbf{w^T x}_n > 0$, we get :

$$\mathbf{w}_{new}^{\mathbf{T}} \mathbf{x}_n = \mathbf{w^T x}_n - ||\mathbf{x}_n||^2$$

Hence, showing that each update to the weight vector nudges the perceptron in the required direction, improving its performance on that particular data-point. The following image illustrates the above discussion :

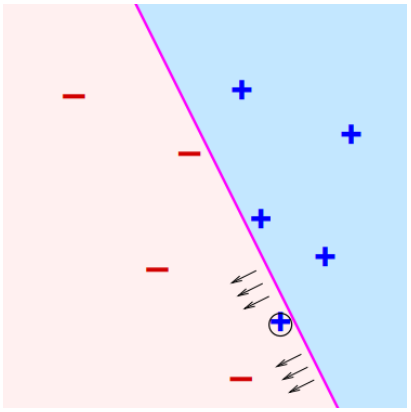

Figure 1: Perceptron

The pink line represents our linear classifier that is uniquely determined by our choice of weights. We have a misclassified **plus** point in the **minus** region and the update rule tries to correct this by nudging the classifier in the required direction.

2.4 Convergence Theorem

The convergence theorem for perceptron, guarantees that if the data is **linearly separable**, repeated updates will eventually classify all points correctly.

Data being linearly separable simply means that, when the data-points are plotted on a two-dimensional graph, it is possible to draw a straight-line that separates them into two distinct categories. The data-points in the above image for example, are linearly separable.

3 Types of Learning

As discussed in the introduction, machine learning is about learning patterns from data. Alternatively, we can put it as **using a set of observations to uncover an underlying process**. In either case, we have established that having a set of observations or data-points is a non-negotiable. However, the form in which the data is available to us can vary. Depending on this form and the nature of the learning task, machine learning is commonly divided into three main paradigms: supervised learning, unsupervised learning, and reinforcement learning.

3.1 Supervised Learning

In supervised learning, the model is trained on labeled data, where each example consists of : (**input, correct output**). The goal is to approximate a function from inputs to outputs that generalizes well to unseen data. For instance, the perceptron model above.

3.2 Unsupervised Learning

In unsupervised learning, the data is in the form of inputs only, without associated labels : (**input, no output**). The task is to discover hidden structures, patterns or clusters within the data. Examples include clustering, dimensionality reduction, and generative modeling.

3.3 Reinforcement Learning

In reinforcement learning, the model interacts with the environment by taking actions and receiving feedback in the form of rewards or penalties : (**action, feedback**). The model learns by trial and error to maximize its cumulative reward, gradually improving its decision-making. This setup is especially suited to sequential problems such as game playing, robotics and control systems.

4 References

1. CalTech Machine Learning Course - CS156, Lecture 1.
2. CalTech Machine Learning Course - CS156, Lecture 4.